

Implementación de la compuerta NAND mediante Hebb, perceptrón y una red de neuronas NOR

Introducción

En este ejercicio se modeló la compuerta lógica NAND con neuronas artificiales. El notebook presenta dos enfoques. Primero, NAND se aprende directamente mediante la regla de Hebb y el algoritmo del perceptrón. Después, la misma función se construye con cuatro neuronas NOR, siguiendo la arquitectura mostrada en la actividad.

La implementación se realizó en Python con PyTorch. Para conservar la misma lógica de los ejercicios anteriores se empleó una representación bipolar: el cero lógico se codificó como -1 y el uno lógico se mantuvo como 1.

1. Fundamento lógico de NAND

NAND significa NOT AND, es decir, la negación de la compuerta AND. Su salida es 1 en tres combinaciones y únicamente produce 0 cuando A y B son iguales a 1.

$$Q = \neg(A \wedge B)$$

A	B	A AND B	A NAND B
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

1.1 Representación bipolar

En los datos del notebook, 0 se transforma en -1. Por ello, el target de NAND se almacena como [-1, 1, 1, 1] cuando las entradas se recorren en el orden (1,1), (1,0), (0,1) y (0,0).

Valor lógico	Valor bipolar
0	-1
1	1

```
S = torch.tensor([
    [ 1, 1],
    [ 1, -1],
    [-1, 1],
    [-1, -1]
], dtype=torch.float32)
```

```
T_NAND = torch.tensor([-1, 1, 1, 1], dtype=torch.float32)
```

2. Modelo neuronal y fórmulas

2.1 Entrada neta

La neurona multiplica cada entrada por su peso, suma las contribuciones y agrega el bias. Este resultado se denomina entrada neta.

$$y_{in} = A \cdot w_1 + B \cdot w_2 + b$$

2.2 Regla de Hebb

La regla de Hebb obtiene los pesos acumulando el producto de cada patrón por su target. En el notebook se utilizó una versión compacta y vectorizada.

$$w(\text{nuevo}) = w(\text{anterior}) + x \cdot t$$

$$b(\text{nuevo}) = b(\text{anterior}) + t$$

```
def entrenar_hebb(X, T):
    w = (X * T[:, None]).sum(dim=0)
    b = T.sum()
    return w, b
```

2.3 Regla del perceptrón

El perceptrón empieza con pesos y bias iguales a cero. Cada patrón se evalúa con la función de activación y, si la predicción es incorrecta, se corrigen los parámetros. En el ejercicio se usaron una tasa de aprendizaje $\alpha = 1$ y un umbral $\theta = 1$.

$$w(\text{nuevo}) = w(\text{anterior}) + \alpha \cdot x \cdot t$$

$$b(\text{nuevo}) = b(\text{anterior}) + \alpha \cdot t$$

```
if y != int(t.item()):
    w += alpha * x * t
    b += alpha * t
    errores += 1
```

3. Representación del funcionamiento de las neuronas

3.1 NAND con una sola neurona

NAND puede representarse directamente con una sola neurona porque sus patrones son linealmente separables. Los pesos obtenidos fueron negativos para ambas entradas y el bias fue positivo.

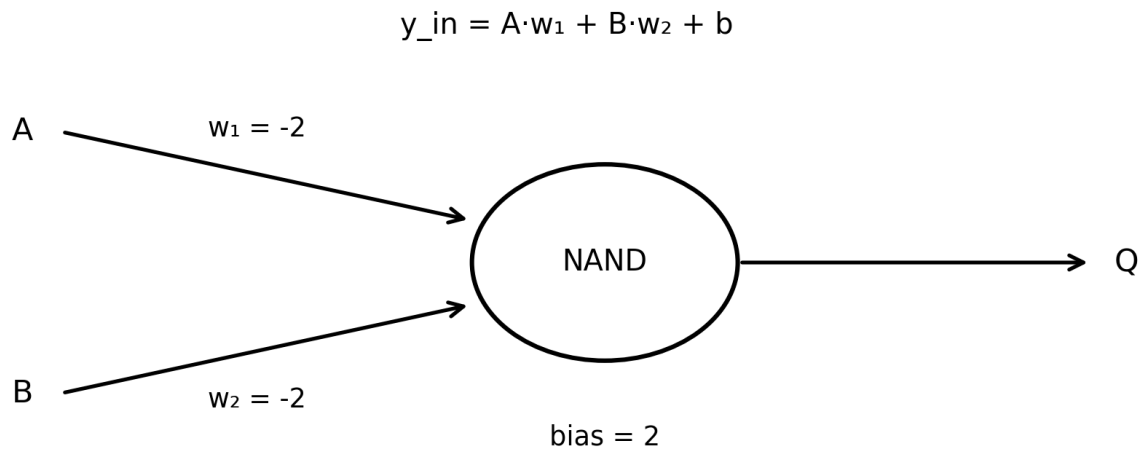


Figura 1. Neurona NAND directa con los parámetros obtenidos.

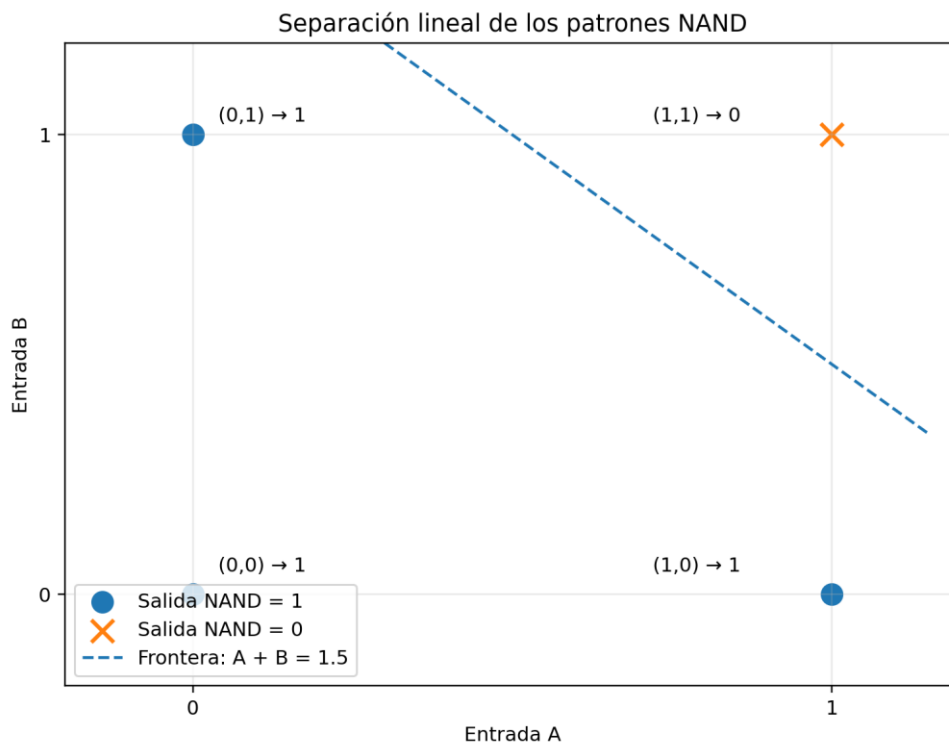


Figura 2. Los patrones NAND pueden separarse mediante una frontera lineal.

3.2 NAND construida con cuatro neuronas NOR

La actividad también pide formar NAND usando únicamente neuronas NOR. Las dos primeras unidades funcionan como inversores, porque reciben dos veces la misma entrada. La tercera reconstruye la operación AND y la cuarta invierte su resultado.

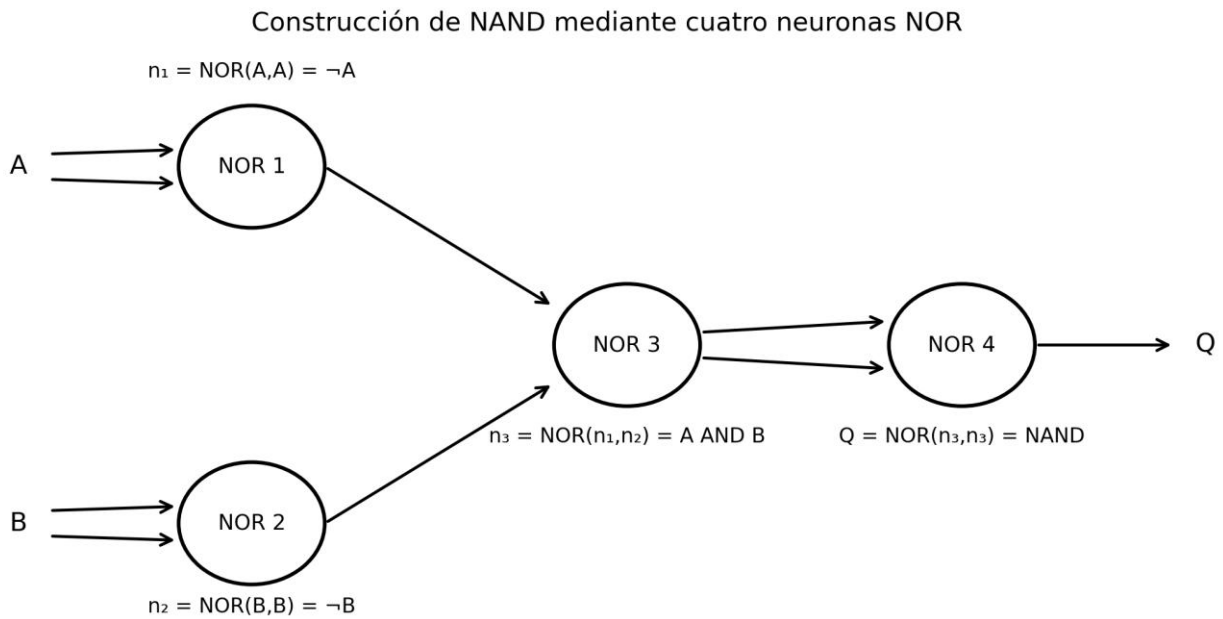


Figura 3. Arquitectura NAND formada por cuatro neuronas NOR.

4. Funcionamiento interno de la red NOR

Operación	Interpretación
$n_1 = \text{NOR}(A, A)$	Produce NOT A.
$n_2 = \text{NOR}(B, B)$	Produce NOT B.
$n_3 = \text{NOR}(n_1, n_2)$	Por De Morgan, produce A AND B.
$Q = \text{NOR}(n_3, n_3)$	Invierte AND y obtiene A NAND B.

```

def nand_desde_nor(x):
    a, b = x.tolist()

    n1 = nor(a, a)    # NOT A
    n2 = nor(b, b)    # NOT B
    n3 = nor(n1, n2)  # A AND B
    q = nor(n3, n3)   # NAND

    return q

```

5. Resultados obtenidos

5.1 Entrenamiento directo de NAND

Método	w_1	w_2	Bias	Umbral	Épocas	Exactitud
Hebb	-2	-2	2	0 implícito	1 recorrido	100 %
Perceptrón	-2	-2	2	1	3	100 %

Los dos métodos llegaron a los mismos pesos y bias. En Hebb, los parámetros se obtuvieron en un recorrido directo por los patrones. El perceptrón necesitó tres épocas para completar el entrenamiento y confirmar que ya no existían errores.

5.2 Validación de la tabla de verdad

A	B	Esperado	NAND directa	Red con NOR
1	1	0	0	0
1	0	1	1	1
0	1	1	1	1
0	0	1	1	1

5.3 Resultados internos de la red de cuatro NOR

A	B	NOT A	NOT B	AND	NAND
1	1	0	0	1	0
1	0	0	1	0	1
0	1	1	0	0	1
0	0	1	1	0	1

La neurona NAND directa y la red formada con cuatro neuronas NOR reprodujeron correctamente las cuatro combinaciones posibles. En ambos casos se obtuvieron 4 aciertos de 4, equivalentes a una exactitud del 100 %.

6. Análisis de los resultados

- Los pesos $w_1 = -2$ y $w_2 = -2$ hacen que las entradas positivas reduzcan la entrada neta.
- El bias positivo $b = 2$ permite que los casos 00, 01 y 10 permanezcan en la clase positiva.
- Cuando $A = 1$ y $B = 1$, la suma ponderada se vuelve negativa y la salida es -1, equivalente al cero lógico.
- NAND no necesita una red multicapa para aprenderse directamente; la red de cuatro NOR se incluyó para respetar la construcción solicitada en la actividad.
- La función vectorizada de Hebb reduce la extensión del código sin modificar la regla matemática.

Conclusión

El ejercicio demostró dos formas válidas de implementar la compuerta NAND. En la primera, una sola neurona aprendió directamente la tabla de verdad mediante Hebb y perceptrón. Los dos métodos obtuvieron $w_1 = -2$, $w_2 = -2$ y bias = 2; el perceptrón trabajó con $\theta = 1$ y terminó en tres épocas.

En la segunda solución se conectaron cuatro neuronas NOR. Las primeras dos generaron NOT A y NOT B, la tercera obtuvo A AND B y la última produjo su negación. Las dos implementaciones alcanzaron una exactitud del 100 %. Además, el uso de funciones compactas permitió conservar la lógica de aprendizaje con un código más corto y fácil de revisar.